NEOVIM / JAVA + AGENT WORKFLOW

Your daily workspace

Open a project. Outline the change. Delegate. Review. Run the tests.

A short daily loop

Space pp opens or returns to a project. Space cs outlines symbols. Space ao asks for a plan. Space an starts a CLI agent. Space gc reviews the diff. Space rt runs tests.

Projects

`Space pd`	Open project directory
`Space pn`	New Project
`Space pp`	Projects
`Space ps`	Save project layout

Files

`Space fS`	Save all files
`Space ff`	Find project file
`Space fg`	Search project text
`Space fn`	New file
`Space fr`	Recent project files
`Space fs`	Save file
`Space e`	File Explorer

Buffers

`Space bb`	Buffer Manager
`Space bd`	Close buffer / hide agent
`Space bl`	Project buffers
`Shift+H / L`	Previous / next buffer

Tabs and views

`Space Tab [`	Previous tab
`Space Tab ]`	Next tab
`Space Tab n`	New view tab
`Space Tab r`	Rename tab
`Space Tab x`	Save and close tab
`gt / gT`	Next / previous tab

Windows

`Ctrl+h/j/k/l`	Move left / down / up / right
`_ / |`	Horizontal / vertical split
`Ctrl+w =`	Equalize split sizes
`Ctrl+w q`	Close a split

Projects stay open

Each project has its own directory and tab. Switching preserves unsaved edits, splits and running language servers. Save/close with Space Tab x to preserve the restart layout.

AI, without losing context

Copilot is the conversation. CLI terminals are the long-running work.

Copilot Chat

`Space aa`	Toggle Copilot Chat [V too]
`Space ao`	AI: outline a change [V too]
`Space ar`	AI: review code [V too]
`Space at`	AI: propose tests [V too]
`Space ae`	AI: explain code [V too]
`Space am`	Choose Copilot model
`Space ap`	Choose Copilot prompt [V too]
`Space ax`	Stop Copilot response
`Space aR`	Reset Copilot conversation

Inside the chat

`Enter [N]`	Send prompt
`Ctrl+s [I]`	Send prompt
`gd [N]`	Preview proposed diff
`Ctrl+y`	Accept proposed diff
`q [N] / Ctrl+c [I]`	Close chat
`Ctrl+l`	Reset conversation
`gh [N]`	Show chat help

Chat context

`#buffer:active`	Active source file (sticky)
`#buffer:listed`	Listed buffers; may span projects
`#file:path`	Attach a specific file
`#gitdiff:staged`	Attach staged changes

Background agents

`Space an`	New CLI agent
`Space al`	All agents / tasks
`Space ah`	Hide agent terminal
`Space as`	Stop agent / task

Worktrees

`Space wb`	Branches (checkout as worktree)
`Space wn`	New branch worktree
`Space ww`	Switch worktree

Shell terminals

`Space tn`	New Terminal
`Space tp`	Pick Terminal
`Space tt`	Toggle Terminal
`Ctrl+/`	Toggle project shell
`Esc Esc [T]`	Terminal to normal mode

One task, one checkout

Use Space wn for a branch/worktree, then Space an inside it. Agents keep the directory where they started. Space al lists jobs from every project; a running process may be waiting for your input.

Lifetime and chat boundaries

Jobs survive project/tab switches while Neovim stays open. Quitting ends them. Save Copilot conversations by name; reset with Space aR when changing topics or projects. Review diffs before accepting edits.

Java, with IntelliJ intelligence

Native Java completion. Explicit builds. Predictable debug controls.

Navigation

`gd / gD`	Definition / declaration
`gI / gr`	Implementation / references
`K / gk`	Documentation / signature
`ge / gE`	Line / cursor diagnostics
`[e / ]e`	Previous / next diagnostic
`Ctrl+o / Ctrl+i`	Jump backward / forward
`Ctrl+Space [I]`	Java completion
`Ctrl+n / Ctrl+p [I]`	Select completion
`Enter [I]`	Accept completion
`za / zM / zR`	Toggle / close / open folds

Code intelligence

`Space cR`	Rename File
`Space cS`	Workspace symbols
`Space ca`	Code action
`Space cf`	Format
`Space ci`	Incoming calls
`Space co`	Outgoing calls
`Space cr`	Rename symbol
`Space cs`	Document symbols / outline

Java and IntelliJ

`Space jR`	Restart IntelliJ server
`Space ji`	IntelliJ health
`Space jl`	IntelliJ license / sign in
`Space jn`	Java file template
`Space jo`	Java import / build output
`Space jr`	Reload Java project

Build and run

`Space rb`	Build project
`Space rd`	Debug Java at cursor
`Space rl`	Run Java code lens
`Space rr`	Run Java at cursor
`Space rs`	Stop Java run / build
`Space rt`	Test project

Debugger

`Space dB`	Conditional breakpoint
`Space db`	Toggle breakpoint
`Space dc`	Continue debugger
`Space de`	Evaluate expression [V too]
`Space di`	Step into
`Space dn`	Step over
`Space do`	Step out
`Space dq`	Stop debugger
`Space dv`	Debug panel
`Space dw`	Watch expression [V too]

Builds use the project wrapper

Space rb uses Maven package -DskipTests or Gradle assemble. Space rt runs the test task. Run/debug at the cursor requires IntelliJ to finish importing and a valid license. Use Space jo to inspect import failures.

Find, inspect, refine

File and text search, source control, editing and display controls.

Search

`Space s"`	Registers
`Space s/`	Search History
`Space sD`	Buffer Diagnostics
`Space sM`	Man Pages
`Space sb`	Grep Open Buffers
`Space sc`	Command History
`Space sd`	Diagnostics
`Space sf`	Smart Find Files
`Space sh`	Help Pages
`Space sm`	Marks
`Space sq`	Quickfix List
`Space ss`	Grep
`Space su`	Undo History

Git review

`Space gL`	Git Log Line
`Space gS`	Git Stash
`Space gb`	Git Branches
`Space gc`	Git Changes (CodeDiff)
`Space gd`	Git Diff (Hunks)
`Space gl`	Git Log
`Space gs`	Git Status

Interface

`Space uH`	Workflow health / prerequisites
`Space ud`	Toggle diagnostic messages
`Space uh`	Toggle inlay hints
`Space um`	Toggle Markdown rendering
`Space un`	Notification history
`Space ?`	Show buffer shortcuts
`Space qq`	Quit Neovim

Editing

`Esc`	Clear search highlight
`gc / gcc`	Comment selection / line
`J / K [V]`	Move selected lines down / up
`< / > [V]`	Indent and retain selection
`Ctrl+v [I]`	Paste clipboard
`Y`	Yank to end of line
`c / C`	Change without replacing yank register
`ysiw"`	Surround word with quotes
`cs"'`	Change double to single quotes
`ds"`	Delete surrounding quotes

A clear visual hierarchy

Tabby: workspace tabs. Lualine: project, branch, diff, diagnostics, LSP and job count. Incline: file labels. Heirline: signs, numbers and folds. hlchunk: active scope and line numbers. Markdown: rendered prose and code blocks.

Setup and command reference

First use needs your accounts. Extra parsers are optional.

Project and agent commands

`:Projects`	Open or return to a project
`:ProjectOpen PATH`	Open any directory as a project
`:ProjectNew`	Create a project under ~/Code
`:ProjectSave`	Save file views / splits / cursors
`:ProjectClose`	Save layout, then close tab
`:Agent [COMMAND]`	Choose an agent or run a command
`:Agents`	Pick a job across all projects
`:AgentStop`	Choose a job to stop

Java commands

`:IntellijLspLicense`	Trial, sign in, activate or inspect license
`:IntellijLspReload`	Reload project after build changes
`:IntellijLspRestart`	Restart language servers
`:IntellijLspOutput`	Import, build and program output
`:IntellijLspFileTemplate`	New Java / Kotlin file template
`:IntellijLspRun[!]`	Run; ! skips build
`:IntellijLspRunStop`	Stop current build or program
`:IntellijLspDebugPanel`	Stack, locals and watches
`:IntellijLspClearCache`	Reindex; use only for broken caches

First launch

Use Neovim 0.12+, Git, curl 8+ and ripgrep. Open :Lazy and restore locked versions. IntelliJ downloads its server, asks for consent and activates through Space jl. Open Space aa for Copilot sign-in. Install/authenticate a CLI agent separately.

Chat and maintenance

`:CopilotChat [PROMPT]`	Ask Copilot
`:CopilotChatSave NAME`	Save conversation locally
`:CopilotChatLoad NAME`	Restore a saved conversation
`:CopilotChatModels`	Choose an available model
`:CopilotChatPrompts`	Choose a prompt
`:CodeDiff`	Git diff review
`:Worktree / WorktreeSwitch`	Branch / worktree picker
`:WorktreeNew`	Create branch and worktree
`:RenderMarkdown toggle`	Toggle Markdown preview
`:Lazy restore`	Install versions in the lockfile
`:Lazy update`	Update plugins; review lockfile diff
`:Mason`	Inspect / install language servers
`:ParsersInstall`	Install optional language parsers
`:TSUpdate`	Update installed parsers
`:checkhealth dotfiles`	Check workflow prerequisites
`:checkhealth intellij-lsp`	Check server and licensing
`:checkhealth CopilotChat`	Check chat dependencies
`:WhichKey`	Discover mappings

What to expect

IntelliJ's Neovim client is pre-alpha and still imports/indexes projects. Its server needs a trial or eligible Ultimate license. Mason installs non-Java servers; some need Node.js. Optional parsers need a C compiler and tree-sitter CLI >= 0.26.1. Keep secrets out of chat context.